Here is the complete project documentation for your multi-product billing and checkout program, structured just like the previous ones!

1. Problem Statement, Objectives, and Assumptions

Problem Statement

In retail and e-commerce checkout systems, manually calculating bills for multiple items, tracking cumulative totals, applying tiered discounts based on purchase thresholds, and calculating sales tax can lead to human calculation errors. An automated checkout system is required to instantly process cart items and generate transparent, itemized bills.

Objectives

- **Automate Multi-Item Cart Processing:** To dynamically collect item prices and quantities for a variable number of products and compute the gross cumulative total.
- **Conditional Threshold Discounting:** To automatically award a 10% discount if the gross purchase total reaches or exceeds a specified threshold (Rs. $1,000$).
- **Tax and Invoice Computation:** To calculate applicable sales tax (5%) on the discounted amount and generate a comprehensive final bill.

Assumptions

- The number of products (n) and item quantities are provided as whole numbers (integers).
- Product prices are provided as numeric floating-point values.
- A 10% discount is unlocked only when the gross total is ≥ 1000.
- Tax is computed at a flat rate of 5% on the subtotal after the discount is applied.

2. Algorithm, Flowchart, and Pseudocode

Algorithm

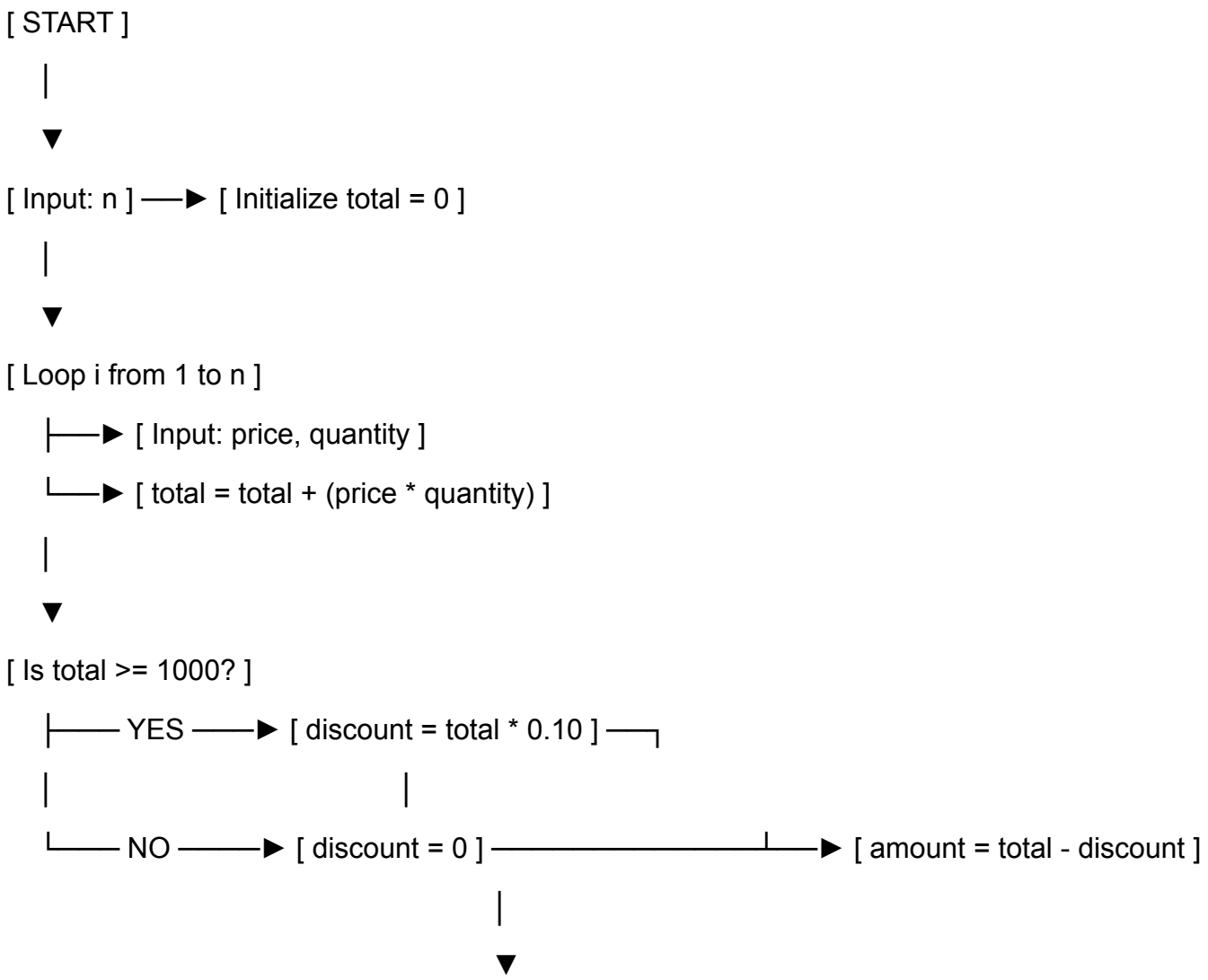
1. **Start** the program.
2. **Input** the number of products n as an integer.
3. **Initialize** total = 0.
4. **Repeat** n times using a loop:

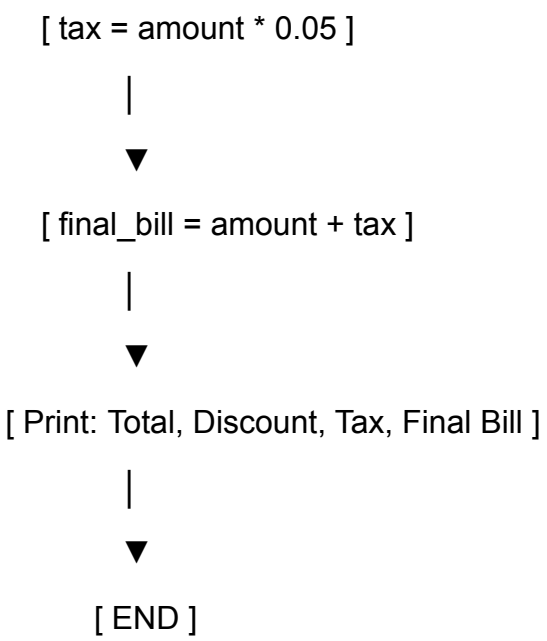
- **Input** price and quantity for the current product.
- Update total = total + (price * quantity).

5. **Evaluate Discount Condition:**
    - If total >= 1000, compute discount = total * 0.10.
    - Else, set discount = 0.
6. **Calculate Discounted Amount:** amount = total - discount.
7. **Calculate Tax:** tax = amount * 0.05.
8. **Calculate Final Bill:** final_bill = amount + tax.
9. **Display Results:** Print Total Amount, Discount, Tax, and Final Bill.
10. **End** the program.

Flowchart Representation

Plaintext





Pseudocode

Plaintext

```
BEGIN
    PRINT "Enter number of products: "
    READ n
    total = 0

    FOR i = 1 TO n DO
        PRINT "Enter product price: "
```

```
    READ price
    PRINT "Enter quantity: "
    READ quantity
    total = total + (price * quantity)
ENDFOR

IF total >= 1000 THEN
    discount = total * 0.10
ELSE
    discount = 0
ENDIF

amount = total - discount
tax = amount * 0.05
final_bill = amount + tax

PRINT "Total Amount:", total
PRINT "Discount:", discount
PRINT "Tax:", tax
```

```
    PRINT "Final Bill:", final_bill
END
```

3. Python Source Code, Test Cases, and Outputs

Python Source Code

Python

```
# Multi-Product Checkout and Billing System

n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))
    total = total + price * quantity

if total >= 1000:
    discount = total * 0.10
else:
    discount = 0
```

amount = total - discount

tax = amount * 0.05

final_bill = amount + tax

print("Total Amount:", total)

print("Discount:", discount)

print("Tax:", tax)

print("Final Bill:", final_bill)

Test Cases Table

Test Case ID	Products Input (Price, Qty)	Gross Total (Rs.)	Discount Applied (Rs.)	Subtotal (Rs.)	Tax (Rs.)	Final Bill (Rs.)	Status
TC_01	(400, 1), (300, 1)	700.0	0.0	700.0	35.0	735.0	Pass
TC_02	(500, 2)	1000.0	100.0	900.0	45.0	945.0	Pass

TC_03	(600, 2), (300, 1)	1500.0	150.0	1350.0	67.5	1417.5	Pass
TC_04	(200, 2)	400.0	0.0	400.0	20.0	420.0	Pass

Simulated Terminal Output

Bash

Enter number of products: 2

Enter product price: 600

Enter quantity: 2

Enter product price: 300

Enter quantity: 1

Total Amount: 1500.0

Discount: 150.0

Tax: 67.5

Final Bill: 1417.5

4. Conclusion

This Python script successfully integrates looping constructs (for), accumulative arithmetic, conditional threshold logic, and tax calculations to simulate a complete retail checkout system. By handling multiple items dynamically and computing itemized totals,

discounts, and taxes transparently, the script provides a robust and scalable prototype for e-commerce cart applications and point-of-sale (POS) software.